Here are **detailed Week 8 notes** for **End-to-End ML Part 2: Session 15 & Session 16**.

Week 8 — End-to-End Machine Learning Part 2

Session 15 & Session 16

1. Main idea of Week 8

Week 8 continues the full **end-to-end temporal machine learning case study**. The main task is to predict **store sales for the next day** using past sales behaviour.

The practical case study focuses on stores `367`, `406`, and `381`, and the task is defined as:

```
Predict next-day store sales using only historical information available before
that day.
```

This is a temporal prediction problem, so the key danger is **information leakage**. The model must not use the target day's sales when building features for that same day.

Part A — Generating Training Data for Time Series

2. What is a training point in time-series prediction?

For next-step prediction, each training example is created using:

```
Input  = historical observations
Output = value at the next time step
```

For example, if sales for one store are:

```
t=0   t=1   t=2   t=3   t=4
10    15    5     20    6
```

Using a history window of 2, we can create examples like:

lag_2	lag_1	output
10	15	5
15	5	20

lag_2	lag_1	output
5	20	6

The uploaded note shows exactly this idea: historical observations become lag features, and the next time step becomes the output.

---

3. Why generate multiple data points?

A time series often has limited data. If we only use the most recent day for each store, we may have too few training examples.

So we generate many supervised learning rows by sliding through time.

Example:

```
Day 100 uses days 99, 93, 70 as lag features.
Day 101 uses days 100, 94, 71 as lag features.
Day 102 uses days 101, 95, 72 as lag features.
```

This converts a time series into a supervised learning dataset.

---

4. Within-store vs across-store training data

There are two sources of training examples:

Within one store

We can create many rows by moving through the store's historical time series.

Benefit:

```
Learns store-specific historical patterns.
```

Problem:

```
Old data may become less useful if the process is non-stationary.
```

Non-stationarity means the data-generating process changes over time. For example, sales behaviour in January may not match sales behaviour in July.

Across stores

We can also use examples from multiple stores.

Benefit:

```
More training data.
```

Problem:

```
Stores may behave differently, and using only one time point across stores may
not capture seasonality.
```

The uploaded note explains that within-store data may be error-prone if older data is no longer predictive, while across-store data may be limited or fail to capture seasonality.

Part B — Creating All Possible ML Data Points

5. Why create all possible data points first?

Instead of manually creating train, validation, and test rows each time, the case study first creates a complete table of all possible supervised learning rows.

This table contains:

```
store_id
day
target value to predict
lag features
rolling-window features
calendar features
```

Then train, validation, and test sets are selected from this table using the day value.

The uploaded note says the workflow is:

```
1. Create a dataframe of all possible data points in SQL.
2. Use get_temporal_Xy() to select the correct points for train, validation, and
test.
```

---

6. Creating a complete daily time series

Raw transaction data is usually an **event series**, not a complete daily time series.

For example, if a store sells nothing on a day, that day may be missing from the transaction table.

But for time-series modelling, missing sales days still matter. A missing day should usually mean:

```
sales = 0
```

not that the day does not exist.

So the notebook creates an `all_days` table with every store-day combination.

Example logic:

```
SELECT store_id, day
FROM all_stores
CROSS JOIN all_days
```

Then it left joins transaction data onto this complete grid.

---

7. Why use LEFT JOIN and COALESCE?

A `LEFT JOIN` keeps every expected store-day row, even if there were no transactions.

`COALESCE(sales_value, 0)` converts missing sales to zero.

Example:

```
SUM(COALESCE(sales_value, 0)) AS sv
```

Meaning:

```
If there was no transaction for this store-day, assume sales were zero.
```

The uploaded SQL note uses this exact logic when building the sales-value time series.

Part C — Feature Engineering for Temporal Sales Prediction

8. Target variable

The target is:

```
to_predict = sales value on the prediction day
```

In SQL:

```
sv AS to_predict
```

So for each store-day row, the model tries to predict that day's sales using information from earlier days.

---

9. Lag features

Lag features use sales from previous days.

Examples:

```
LAG(sv, 1)  OVER (PARTITION BY store_id ORDER BY day) AS sv_1
LAG(sv, 7)  OVER (PARTITION BY store_id ORDER BY day) AS sv_7
LAG(sv, 30) OVER (PARTITION BY store_id ORDER BY day) AS sv_30
```

Interpretation:

Feature	Meaning
`sv_1`	Sales yesterday
`sv_7`	Sales same day last week
`sv_30`	Sales around one month ago

These are useful because sales often have daily, weekly, or monthly patterns.

---

10. Rolling-window features

Rolling-window features summarize recent history.

Example:

```
SUM(sv) OVER (
    PARTITION BY store_id
    ORDER BY day
    ROWS BETWEEN 7 PRECEDING AND CURRENT ROW
) - sv AS sv_w1
```

This creates sales in the previous 7 days.

The `- sv` is very important.

Without subtracting `sv`, the rolling sum would include the current day's sales, which is the target. That would be leakage.

The uploaded note explicitly warns that if we do not subtract `sv`, it becomes an information leak.

---

11. Monthly rolling-window feature

Similarly:

```
SUM(sv) OVER (
    PARTITION BY store_id
    ORDER BY day
    ROWS BETWEEN 30 PRECEDING AND CURRENT ROW
) - sv AS sv_m1
```

This gives sales over the previous month, excluding the target day.

Interpretation:

```
sv_m1 = recent monthly sales history before the prediction day
```

---

12. Calendar features

The case study also creates approximate calendar features:

```
day % 364 AS woy
day % 30  AS wom
```

Feature	Meaning
`woy`	Approximate week-of-year / yearly cycle
`wom`	Approximate within-month cycle

These help the model learn seasonal patterns.

---

13. Why filter only after window functions?

The SQL note includes this important point:

```
Restrict after any window function that looks into the past.
```

If we filter too early, the window functions cannot see earlier rows.

Example mistake:

```
WHERE day >= 95
```

before calculating `LAG(sv, 30)`.

Then day 95 cannot access day 65 because it has already been removed.

Correct approach:

```
1. Build full time series.
2. Calculate lag and rolling features.
3. Then filter rows where enough history exists.
```

The uploaded note says the subquery is needed because the restriction must happen after the window functions; otherwise the windows cannot access past data.

Part D — Temporal Train / Validation / Test Splitting

14. Why temporal splitting is needed

In a temporal ML problem, validation and test data should be in the future relative to training data.

The uploaded note explains:

```
Test and validation data are data points where we pretend not to know the
output,
selected to be temporally in the future of the training set.
```

So we cannot use random train-test splits.

---

15. Single train / validation / test split

The notebook first uses a simple split:

```
Train day:      709
Validation day: 710
Test day:       711
```

This means:

```
Train on earlier day(s)
Validate on the next day
Test on the most recent held-out day
```

The reason for using the most recent day as test is that it is closest to the real future deployment setting.

---

16. Baseline model

The first baseline is simple:

```
Prediction for today = sales yesterday
```

In feature terms:

```
baseline = X_valid.sv_1.to_numpy()
```

This is a **naive temporal baseline**.

It is important because a complex model is only useful if it beats a simple reasonable baseline.

---

17. First model: Linear Regression

The first basic model is linear regression:

```
from sklearn.linear_model import LinearRegression

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred = lr.predict(X_valid)
```

This gives a simple benchmark ML model.

The evaluation compares:

```
Baseline error
Linear regression error
```

If the ML model cannot beat the baseline, then the modelling approach or features need improvement.

Part E — Using More Historical Training Data

18. Why one training day is not enough

Training on one day gives very few rows, especially with only three stores.

For example:

```
3 stores × 1 day = 3 training rows
```

That is far too small for most ML models.

So the notebook extends the training set by using many historical reference days.

Example:

```
train_ref_days = range(train_ref_day - 100, train_ref_day + 1)
```

This means the model trains on about 100 previous daily datasets.

---

19. Updated temporal data selection function

Instead of selecting one reference day, the notebook updates the function so it can select multiple days:

```
def get_temporal_X_y(ref_days, debug=False):

    if isinstance(ref_days, int):
        ref_days = [ref_days]

    sql = f"""
    SELECT *
    FROM all_ml_data_pts
    WHERE day IN ({','.join([str(x) for x in ref_days])})
    """

    Xy = spark.sql(sql).toPandas()
    X = Xy.drop(columns=["to_predict"])
    y = Xy.to_predict.to_numpy()

    return X, y
```

This allows training data to be built from many historical days while validation remains a future day.

Part F — Repeated Train / Validation Splits

20. Why repeated validation is used

A single validation day may be misleading.

Sales can vary due to:

```
day-of-week effects
store-specific behaviour
promotions
```

```
seasonality
random noise
```

So the notebook creates repeated temporal validation splits.

Example:

```
Validation day 710, train on previous 100 days
Validation day 709, train on previous 100 days
Validation day 708, train on previous 100 days
...
```

This gives a more stable estimate of performance.

---

21. How repeated temporal validation works

For each validation reference day:

```
Validation set = rows for that day
Training set = rows for previous n days
```

Example:

```
Validation day = 710
Training days  = 609 to 709
```

Then move backwards:

```
Validation day = 709
Training days  = 608 to 708
```

This is still temporal because every validation day is after its training window.

---

22. Why scikit-learn needs index splits

`cross_val_score` and `GridSearchCV` expect `cv` to be a list of index pairs:

```
(train_indices, validation_indices)
```

So the notebook creates an indexed version of the ML data table:

```
CREATE OR REPLACE TABLE all_ml_data_pts_with_index AS
SELECT
    row_number() OVER (ORDER BY store_id, day) - 1 AS idx,
    *
FROM all_ml_data_pts
ORDER BY store_id, day
```

The `-1` is used because SQL row numbers start at 1 but pandas indices start at 0.

---

23. Creating temporal CV indices

The notebook then creates functions to return indices for selected days:

```
def get_temporal_X_y_idxs(ref_days, debug=False):

    if isinstance(ref_days, int):
        ref_days = [ref_days]

    sql = f"""
    SELECT idx
    FROM all_ml_data_pts_with_index
    WHERE day IN ({','.join([str(x) for x in ref_days])})
    """

    return spark.sql(sql).toPandas().idx.to_numpy()
```

And for a train-validation split:

```
def get_train_valid_set_idxs(ref_day=710):

    valid_idxs = get_temporal_X_y_idxs(ref_day)

    train_ref_days = range(ref_day - 101, ref_day)
    train_idxs = get_temporal_X_y_idxs(train_ref_days)

    return train_idxs, valid_idxs
```

This allows proper temporal validation inside sklearn.

Part G — Model Refinement

24. What happens after baseline evaluation?

After the first evaluation, the notebook asks what can be improved:

```
Extra features?
Different preprocessing?
Different features?
Better models?
Better hyperparameter tuning?
```

This is the refinement stage of end-to-end ML.

---

25. Hyperparameter tuning with GridSearchCV

Once temporal CV indices are created, `GridSearchCV` can be used safely.

Example:

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestRegressor

grid = {
    "max_depth": [4, 5, 6, 7, 8, 9, 10, 15, 20, 25, 30, 35, 40],
    "max_features": [1, "log2", "sqrt"]
}

rf = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

rfgs = GridSearchCV(
    rf,
    grid,
    cv=all_cv_indexes,
    scoring="neg_mean_absolute_percentage_error"
)

rfgs.fit(X, y)
```

Important point:

```
GridSearchCV is only valid here because cv uses temporal train-validation
splits.
```

Random CV would be wrong.

---

26. Random Forest model

Random Forest is used as a stronger non-linear model.

Why use it?

```
Handles non-linear relationships
Captures interactions
Works with mixed feature scales
Usually strong baseline for tabular data
```

But it still needs careful temporal validation.

---

27. LightGBM model

The notebook also tries a LightGBM regressor:

```
from lightgbm import LGBMRegressor
```

LightGBM is a gradient boosting model and often performs well on tabular datasets.

The tuned parameters include:

```
learning_rate
max_depth
feature_fraction
subsample
n_estimators
```

This is part of the “try a better model” step.

Part H — Adding More Features

28. Why add more features?

Initial features may not fully capture store behaviour.

So the notebook adds extra variables, including visit-based features.

Example new raw signal:

```
COUNT(DISTINCT basket_id) AS visits
```

This allows the model to understand not only sales value, but also store traffic / number of baskets.

---

29. Visit lag features

Just like sales lags, the notebook can create visit lags:

```
visits_1
visits_2
visits_7
```

These may help because sales can be driven by number of visits as well as basket value.

---

30. Feature importance and feature reduction

After adding features, the notebook uses **permutation importance** to identify which features matter.

The idea:

```
1. Train model.
2. Measure validation performance.
3. Shuffle one feature.
4. Re-measure performance.
5. If performance gets worse, the feature was important.
```

This helps decide whether some features can be removed.

---

31. Manual feature elimination

The notebook manually removes some less useful features:

```
X_reduced = X.drop(columns=[
    "sv_w1",
    "sv_m1",
    "sv_30",
    "day",
    "wom",
    "visits_1",
    "visits_2"
])
```

Important caveat:

```
Manual feature removal is not always ideal, especially when interactions exist.
```

A feature may look weak alone but matter in combination with another feature.

Part I — Final Test Evaluation

32. Why keep a held-out test set?

Validation is used for model selection and tuning.

The test set should be used only at the end to estimate expected performance on new data.

In the notebook:

```
test_idxs = get_temporal_X_y_idxs(711)
```

Then:

```
X_test = X_reduced.iloc[test_idxs]
y_test = y[test_idxs]
```

Training uses all non-test rows:

```
X_train = X_reduced.drop(test_idxs, axis=0)
y_train = np.delete(y, test_idxs, axis=0)
```

Then final performance is measured on the test day.

---

33. Why test evaluation is “tenuous” here

The notebook notes that using one time slice as the test set is somewhat weak because there are only three stores.

That means:

```
Test set = 3 rows
```

So the final estimate may be noisy.

In a real project, we would prefer more stores, more test days, or a longer temporal hold-out period.

Part J — Evaluation Metrics

34. MAE

Mean Absolute Error:

```
Average size of prediction error in original units.
```

Example:

```
If sales are in £, MAE is in £.
```

Benefit:

```
Easy to interpret.
```

---

35. MAPE

Mean Absolute Percentage Error:

```
Average percentage error.
```

Formula idea:

```
absolute error / actual value
```

Benefit:

```
Easy to compare across stores of different size.
```

Problem:

```
Unstable or undefined when actual sales are zero or very small.
```

---

36. Why compare to baseline?

Always compare ML models to a simple baseline.

For next-day sales, a strong simple baseline is:

```
Tomorrow’s sales = today’s sales
```

If a complex model cannot beat this, it is not useful.

Part K — Model Understanding at the End

37. Why interpret the final model?

After selecting a model, we want to understand what it learned.

The notebook uses:

```
Permutation importance
Partial dependence plots
SHAP values
```

These help explain which features matter and how they affect predictions.

---

38. PDPs in the final model

The notebook creates PDPs for features such as:

```
sv_3
sv_1
woy
```

Interpretation examples:

```
sv_1: how yesterday's sales affect predicted sales
woy: how approximate seasonality affects predicted sales
```

A PDP shows how the model's average prediction changes as a feature changes.

---

39. 2D PDP interaction

The notebook also creates an interaction PDP:

```
feats = ["sv_1", "woy"]
```

This checks whether the effect of yesterday's sales depends on the seasonal position in the year.

Example interpretation:

```
Yesterday's sales may be more predictive in some parts of the year than others.
```

---

40. SHAP summary plot

The notebook computes SHAP values:

```
import shap

shap_values = shap.TreeExplainer(lgbm).shap_values(X_reduced)
shap.summary_plot(shap_values, X_reduced)
```

SHAP helps explain how features push predictions up or down.

In this context, SHAP can show:

```
Which features increase predicted sales
Which features decrease predicted sales
Which features matter most overall
```

Part L — Strong Exam Points

41. Why is random CV invalid here?

Random CV would mix past and future rows.

That means the model might train on examples from after the validation/test period. This gives an unrealistic estimate of future performance.

Correct approach:

```
Use temporal CV where validation is always after training.
```

---

42. Why create `all_ml_data_pts` first?

Because it separates two tasks:

```
Feature generation
Temporal dataset selection
```

First, create every possible valid supervised learning row. Then select train, validation, and test rows by time.

This makes the workflow cleaner and reduces mistakes.

---

43. Why subtract `sv` from rolling sums?

Because `sv` is the target for the current row.

If we include it in the rolling sum, the model indirectly sees the answer.

Correct:

```
SUM(sv) OVER (...) - sv AS sv_w1
```

Incorrect:

```
SUM(sv) OVER (...) AS sv_w1
```

The incorrect version leaks target information.

---

44. Why use multiple validation folds?

A single validation day may not represent general performance.

Repeated temporal validation tests the model across several prediction dates while preserving time order.

This gives a better estimate of how the model performs over time.

---

45. Why use a final held-out test day?

Because validation performance is used to choose features, models, and hyperparameters.

The test set gives a final unbiased estimate of expected future performance, as long as it was not used during model selection.

---

46. End-to-end ML workflow from Week 8

```
1. Define the prediction task
   Predict next-day sales for selected stores.

2. Explore the data
   Check date ranges, missing days, and store coverage.

3. Create a complete time series
   Build all store-day combinations.

4. Fill missing sales
   Use LEFT JOIN and COALESCE to represent no-sales days as zero.

5. Generate temporal features
   Create lag, rolling-window, visit, and calendar features.

6. Avoid leakage
```

```
   Exclude current-day target from rolling features.

7. Create all possible ML data points
   Store each day-store supervised row in one table.

8. Build temporal splits
   Train on past days, validate on future days, test on final held-out day.

9. Build baselines
   Use yesterday's sales as a naive baseline.

10. Train simple models
   Start with linear regression.

11. Use repeated temporal validation
   Evaluate across multiple reference days.

12. Tune stronger models
   Random Forest and LightGBM with GridSearchCV using temporal CV indices.

13. Add and reduce features
   Add visit features, use permutation importance, remove weak features
carefully.

14. Final test evaluation
   Estimate expected predictive performance on held-out future data.

15. Interpret the model
   Use permutation importance, PDPs, interaction PDPs, and SHAP.
```

---

47. Likely exam-style answers

Q1. How do we turn a time series into supervised learning data?

We create lagged input features from historical observations and use the next time step as the output. For example, using a two-day history, sales at `t−2` and `t−1` become input features and sales at `t` becomes the target. Repeating this across days and stores creates a supervised learning dataset.

Q2. Why do we create all possible data points before splitting?

Creating all possible data points first gives a clean pool of valid temporal examples. Train, validation, and test sets can then be selected by reference day. This makes it easier to enforce temporal ordering and avoid leakage.

Q3. Why is `COALESCE` used when building the sales time series?

`COALESCE` is used to replace missing sales values with zero. If a store-day has no transaction rows, this usually means the store sold nothing on that day, not that the day should be removed. This creates a complete daily time series.

Q4. Why must rolling features exclude the current day?

The current day's sales are the target variable. If a rolling feature includes the current day, the model indirectly sees the answer. Subtracting the current day's value from the rolling sum prevents information leakage.

Q5. How does repeated temporal validation work?

Repeated temporal validation creates multiple train-validation splits where each validation day is after its training window. For example, train on days 609–709 and validate on day 710, then train on 608–708 and validate on 709. This evaluates performance across multiple temporal reference points while preserving time order.

Q6. Why can GridSearchCV be used in this temporal case?

GridSearchCV can be used only because we pass custom temporal CV indices. These indices ensure that each validation set is in the future relative to its training set. Without custom temporal indices, normal random cross-validation would be inappropriate.

Q7. Why compare LightGBM or Random Forest against a baseline?

Complex models should only be used if they outperform simple, reasonable baselines. In temporal sales prediction, yesterday's sales is a natural baseline. If Random Forest or LightGBM cannot beat it, the extra complexity is not justified.

Q8. What is the final role of PDP and SHAP?

PDPs show how the model's average prediction changes as a feature changes, such as how yesterday's sales affects predicted sales. SHAP values explain how features contribute to individual predictions and provide an overall summary of feature influence. Both explain the model, not necessarily the true causal process.

---

48. One-page revision summary

```
Week 8 focuses on completing an end-to-end temporal ML workflow.

Task:
Predict next-day store sales.

Core idea:
```

Convert time series into supervised rows using lagged history as inputs and next-day sales as output.

Feature creation:
Use lag features, rolling-window features, visit features, and calendar features.

Key leakage warning:
Do not include the current day’s sales in features. Rolling windows must subtract sv.

Data preparation:
Create all store-day combinations, left join transactions, and use COALESCE to treat missing sales as zero.

Evaluation:
Random splitting is invalid. Use temporal train-validation-test splits.

Validation:
Use repeated temporal validation by moving the validation reference day backward and training only on earlier days.

Modelling:
Compare naive baseline, linear regression, random forest, and LightGBM.

Tuning:
GridSearchCV is valid only with custom temporal CV index splits.

Refinement:
Add features, use permutation importance, and reduce features carefully.

Final test:
Use the held-out future day only once for expected predictive performance.

Understanding:
Use permutation importance, PDPs, 2D PDPs, and SHAP to interpret the final model.